

## Presenter Reference Notes – September 2026

Generated from the revised deck. These notes preserve slide claims and evidence limits; they are not an independent evidence source.

### Slide 1: NDNSF: A Data-Centric Service Framework for

Secure, Collaborative, and Dynamic Networked Applications Department of Computer Science University of Memphis Advisor: Dr. Lan Wang PhD Proposal Defense September 2026

### Slide 2: Why Service-Level Mechanisms Are Needed

Networked applications need mechanisms for service discovery, invocation authorization, and application-data exchange. Some operations discover a currently willing Provider; others must retain an already selected Provider. Applications must associate permissions, execution state and exchanged data with the requested operation. Changes in service availability, resources and connectivity add further challenges. Research goal: a reusable service protocol. UAV and inference applications provide validation cases, not limits on the framework's intended scope.

### Slide 3: From Invocation to Collaboration: Two Research Scopes

Foundation: evaluate discovery and authorization The initial prototype discovers candidates and selects one Provider whose positive ACK passes validation. Discovery confidentiality, service permissions and management costs matter even without collaboration. Extension: coordinate complementary roles Some tasks need several Providers with different responsibilities. Separate calls do not define a shared role assignment or enforce dependencies between them. Invocation and initial plan-commit mechanisms exist. [28] The proposed research evaluates authorization choices, completes dependency/recovery rules, and tests their reuse and cost.

### Slide 4: NDN Primitives and Service-Level Decisions

NDN building block Remaining application responsibility Named Data and signatures [1] Apply Trust Schema [16] rules to validate signing authority Interest retrieval and caching [1,26] Check task identity, dependencies and deadlines; caches do not guarantee storage Dataset synchronization / SVS [2,3] Discover names; define service-message transitions NAC [4] / enhanced NAC-ABE [19] Encrypt Content for read access; check invocation permission separately NDN: Named Data Networking [1]; NAC: Name-based Access Control; ABE: attribute-based encryption; SVS: State Vector Sync. NDNSF adds service authorization and invocation-state rules.

### Slide 5: Research Scope: From Application Needs to Checks

Follow the invocation lifecycle, informed by invocation [9,10] and authorization [12] mechanisms. Stage / concern Failure to test RQ Discover candidates Ineligible candidate or unauthorized descriptor access 1, 3 Select execution Wrong Provider, unauthorized or repeated start 1, 3 Use inputs Wrong producer, object or dependency 1–3 Close or replace Late results advance the wrong execution 2, 3 Change permissions Missed grant; withdrawn permission still accepted 1, 3 Test applicable rows in the declared UAV and inference workflows. Permission changes affect multiple stages.

## Slide 6: Three Research Questions

RQ1: When does ABE permission reuse offer a better cost-protection tradeoff than complete role-authorized alternatives, as permission scale, overlap, membership and withdrawals change? RQ2: How can single-Provider invocation be extended to roles and named dependencies across Providers, whether Data comes from producers or caches? RQ3: When do these mechanisms improve completion, latency, or execution efficiency, and when do their costs outweigh the benefits? RQ1 studies invocation; RQ2 studies collaboration. RQ3 evaluates their benefits and costs, not another protocol.

## Slide 7: Related Work: Mechanisms and Limits of the Comparison

Published mechanism Proposed comparison boundary NSC [10]: signed invocation Caller authorization versus task-specific predecessor checks DNMP [12]: role commands Standing permission versus this task's assignment CFN [33]: dynamic graph Internally trusted components versus explicit object-admission rules CaSCON [15]: composition Caching is prior art; detailed recovery rules still need verification NFN [6], NFaaS [7] and RICE [9] are computation/invocation precedents. Proposed focus: turn post-ACK assignments into input-acceptance rules, including explicit artifact reuse during replacement.

## Slide 8: Research Hypothesis and Expected Contributions

Hypothesis: A common named-Data protocol can separate discovery from authorized execution and enforce runtime role assignments when intermediate objects are used. Intended knowledge Evidence to deliver When permission reuse is worthwhile Permission overlap, churn and lifecycle costs under equal protection Which task-specific checks are needed Valid completion; wrong-role/edge and replacement tests Which rules can be shared Common Core checks; application validators; matched utility tests Overall contribution: an evaluated service framework for invocation, named-object exchange and collaboration across UAV and inference applications.

## Slide 9: Participants and Trust Bootstrap

User: requests a service and selects a Provider; the extension also assigns collaboration roles. Provider: offers and executes a service. A node may play both roles. Controller: administers identities and service permissions approved by the organization. Participants use provisioned trust anchors and validation rules for certificates and Controller material. Enrollment establishes identities and service permissions. The User then discovers which authorized Providers are currently willing and selects participants for the task.

## Slide 10: Invocation Overview: Request a Timely Aerial Image

An operator requests an aerial image. Authorized camera Providers A and B consider the task. User Request Data Candidates ACK Data User Selection Data Selected Provider Response Data A returns a positive ACK; B is busy and returns a negative ACK. The User selects A; A checks Selection before capturing the image. Arrows show application order. Data is retrieved with Interests; neither an ACK nor a network delivery alone authorizes execution.

## Slide 11: Single-Provider Invocation: Four Named Data Types

Data Logical purpose Request Describe required service and participation constraints; include protected discovery information and RequestNonce ACK Report positive or negative participation; positive ACK

supplies selection information and ProviderChallenge Selection Assign selected execution; provide input/references and protected request-scoped keys; constructed using ProviderChallenge Response Return status, results or result references; the User still validates them before use Sync helps discover names; Interests retrieve Data. The Provider checks Selection before execution; a positive ACK alone does not start the task.

## **Slide 12: Request: Discovery Descriptor; Selection: Task Input**

Discovery descriptor Needed to decide whether to participate: area, deadline, sensor/model and resource requirements. Application input Needed to execute the selected task: images, detailed processing parameters, or protected input references. Input and results use request-scoped symmetric keys. Selection delivers key material encrypted for the selected Provider; the requesting User retains the keys. The boundary follows necessity. A location needed to assess coverage belongs in the protected descriptor; an image needed only for processing can wait.

## **Slide 13: Three Security Requirements; One Additional Benefit**

1. Keep discovery descriptors confidential from unauthorized readers before selecting a Provider.
2. Authenticate participants, verify service permissions, and reject replayed invocation messages.
3. Grant and withdraw service permissions during operation. Additional benefit: aggregate service permissions in a decryption key (DKEY) while retaining identity-signing credentials. This is a management property, not another security guarantee.

## **Slide 14: Reason 1: Confidential Discovery Before Selection**

The User knows the service, but not yet the currently willing Provider identities. An observation area and deadline can reveal intended activity even without the full application input. The User encrypts the descriptor using ABE and the service attribute, without listing Provider identities. [19] Confidential from parties lacking the required decryption capability. Authorized but unselected Providers can read it; names and traffic patterns remain visible.

## **Slide 15: Reason 2: Reuse Service-Permission Management**

ABE was introduced to keep discovery descriptors non-public. [19] NDNSF reuses the attribute authority to manage Provider and User service permissions. [28] RequestNonce and ProviderChallenge checks use these permissions during invocation. ABE discovery plus role-based invocation is also possible. NDNSF avoids adding role signing credentials solely for service membership; signature validation and request-state checks remain necessary. User DKEYs and challenge processing still add cost.

## **Slide 16: How NDNSF Checks Service Permissions**

1. Provider permission: recover RequestNonce from ABE-protected Request content and return it in ACK Data. The User compares it with RequestNonce stored for this request.
2. User permission: recover ProviderChallenge from ABE-protected ACK content and construct Selection Data. The Provider validates Selection using its stored ProviderChallenge. Participants also check current service permissions. Challenge checks establish access to protected values, not exclusive key possession. RequestNonce is an application value, distinct from the Interest Nonce.

## Slide 17: Identity and Replay Checks Have Different Jobs

Identity authentication Validate Data signatures and certificate chains under trust rules. An accepted signing identity alone does not establish service permission. Replay protection Match messages to an active request and its state. Record whether ProviderChallenge has already been used to accept Selection for execution; duplicate Selection must not start another execution. Retain or safely invalidate request state, including across restart. Encryption or a challenge value alone does not prevent replay.

## Slide 18: DNMP: Trust-Schema Command Authorization

Distributed Network Measurement Protocol (DNMP) [12] authorizes measurement commands: 1. The Network Key anchors role keys, device keys, and the schema. 2. The client uses target, probe type, arguments, and user role to find an allowed signing key; among matching keys, it chooses the least privileged. 3. The receiving Network Observer Daemon validates the publication under the schema before dispatching the probe. DNMP also uses publication lifetimes and retained state against replay, assuming loosely synchronized clocks. These are concrete command-authorization mechanisms, not just identity checks.

## Slide 19: DNMP-inspired Authorization versus NDNFSF

Design Execution authorization Discovery confidentiality DNMP-inspired [12] Check role-constrained signatures and NDNFSF invocation state Retain ABE for the first controlled comparison NDNFSF ABE-backed Check signatures, RequestNonce, ProviderChallenge, current permissions and request state User encrypts descriptor content with service-level ABE Hold discovery encryption constant to isolate authorization cost. Both retain Selection, current permissions and replay checks. This adapts DNMP, not its original implementation. Group encryption is a separate comparison.

## Slide 20: RQ1: Evaluate Authorization Design Tradeoffs

For  $n$  Users and  $k$  service permissions per User, on average: Additional authorization credentials Logical objects One role certificate per service  $nk$ :  $O(nk)$  certificates One policy DKEY per User  $n$ :  $O(n)$ , but policy size grows Both retain identity certificates. Shared roles can also use  $n$  certificates. Fixed  $k$ : both grow linearly with  $n$ . Fewer credentials need not cost less. Measure CPU time and bytes for setup, calls, permission updates and background checks.

## Slide 21: RQ1: Three Planned Validation Configurations

Configuration Controlled comparison G1: Per-service roles 100 Users, five permissions each: 500 role certificates versus 100 DKEYs. G2: Shared combination roles Same permissions: 100 role certificates versus 100 DKEYs. Shared roles, individual private keys. G3: Frequent withdrawals Fixed invocation rate; matched grant/withdrawal traces. Compare update delay, rekey traffic and completion. Analytical examples, not measurements. Both retain ABE-protected discovery, identical permissions and the same status-validity bound. Identify when each design is preferable, with correctness and availability checked alongside cost.

## Slide 22: Runtime Permission Grants and Withdrawals

Both operations update Controller-signed permission status. [28] Ordinary grant: update the target's complete policy; it retrieves a replacement DKEY. Public parameters stay unchanged. Withdrawal:

rotate the Controller's ABE master key and public parameters; remaining members obtain DKEYs for their remaining permissions. The master key stays with the Controller. Rotation is Controller-wide. Participants must retrieve and validate updates before applying them; disconnected nodes cannot do so immediately.

## **Slide 23: Permission Updates: Discovery, Validation and Application**

1. Revised design: the Controller publishes new public parameters as signed Data; Sync announces availability. 2. Participants retrieve and validate parameters and permission status, then refresh DKEYs as required. 3. Grants also require status discovery, even when public parameters do not change. Status retrieval and rotation are implemented; Sync announcements are planned for end-to-end validation. Evaluation will measure propagation, rejection delay and rekeying traffic. Previously disclosed keys and plaintext remain accessible to their holders.

## **Slide 24: Threat Model: What Is Trusted?**

Trusted: Controller authorization and the User's planning decision. Untrusted delivery: an object may arrive through a producer, forwarder, cache, or repository and still requires validation. Adversarial cases to test: wrong identities, permissions, challenges, request state, roles, dependencies, or execution identities. Not established by signatures/ABE: honest resource reports, correct inference, truthful images, or resistance to authorized key sharing. Scope: trusted Controller and User, Content confidentiality, and withdrawal after validated updates. Malicious-coordinator agreement and traffic-analysis resistance are outside this scope.

## **Slide 25: RQ2 Extension: From Positive ACKs to a Role Plan**

1. Application supplies roles, dependencies, constraints and deadline. 2. The User closes ACK collection and freezes eligible, validated positive ACKs. 3. A replaceable policy assigns Providers; the User signs the plan. 4. Providers validate assignments, permitted inputs and resource readiness. The template precedes discovery; concrete signer/name constraints follow ACKs. Core commit checks exist. [28] Dependency and recovery contracts remain research to validate.

## **Slide 26: A Plan Is More Than a Request ID**

Proposed task: User assigns UAV A to view 1, UAV B to view 2, and Provider C to analysis. UAV A signed ViewImage UAV B signed ViewImage C validates producer and edge C analyzes after both inputs pass Arrows show object dependencies; Interests retrieve the images from producers or caches. B's valid signature and matching request ID do not let its image fill A's view-1 role. C must check the assignment before accepting that dependency.

## **Slide 27: Plan-to-Data Enforcement: Check Before Advancing**

1. Validate the Data signature using the appropriate trust rules. 2. Check active execution and deadline; identify the permitted dependency. 3. For new Data, match request/plan/execution and assigned producer; for historical Data, check explicit reuse permission. 4. Check that the object matches the declared edge, permitted name, data format and key scope. 5. Require complete validated input before the dependent computation starts. The proposed rules apply equally to Data retrieved from producers,

caches and repositories. End-to-end tests will check valid inputs and rejection of wrong-role, wrong-edge and outdated-execution objects.

## **Slide 28: Collaboration Recovery Needs Explicit State**

Event Required behavior Late ACK Do not silently change the committed plan Duplicate Selection Check whether execution was already authorized; do not repeat a side effect Incomplete dependency Wait within policy/deadline, recover, or fail; do not execute on partial data Provider replacement Authorize a new assignment; reject old execution actions and specify permitted result reuse Cancellation Prevent late messages from reauthorizing cancelled work A new plan may permit a specific old artifact with its original name and signature. This does not reactivate the old execution. Application checks still decide whether it is usable.

## **Slide 29: NDNSF-UAV: Planned Framework Validation**

UAV (unmanned aerial vehicle) workflows test NDNSF mechanisms. [28] Application function NDNSF validation scope Control, telemetry, mission status Provider/request association, permissions, operation lifecycle Live video and recordings Stream lifecycle, protected delivery, named-object retrieval Detection and multi-view tasks Service provision/consumption, role assignments, dependencies Use these workflows to test the invocation foundation and collaboration extension. Validate service behavior through the actual NDNSF path; evaluate flight safety and perception accuracy separately.

## **Slide 30: UAV Collaboration Example: Capture, Analyze and Return**

An operator selects a ground area on a map and requests views and analysis. 1. Selected UAVs capture views, symmetrically encrypt images, and publish signed ViewImage Data under their own authorized namespaces. 2. The assigned compute Provider retrieves permitted images and publishes Annotation Data. 3. Its encrypted result supplies original image names and permitted keys; the requester retrieves images and displays annotations. Reuse original image Data. Fixed-camera viewing geometry requires synchronized UAV attitude and calibration; GPS alone does not establish visibility.

## **Slide 31: UAV Authorization and Safety: Separate Responsibilities**

Core: service authorization and request state. UAV application: operator authority, mission progress and reassignment. Vehicle: readiness, flight limits and local failsafe behavior. Operator leases are currently checked at the ground station, without an independent repeat check in the Drone MAVLink handler. [28] Tests will examine this command path. NDNSF ACK reports willingness; flight-controller acceptance and vehicle behavior are separate observations.

## **Slide 32: NDNSF-DI: Planned Framework Validation**

Existing native lifecycle: prepare a model, submit an inference request, and collect results. [28] Application function NDNSF validation scope Single-Provider inference Discovery, permissions, input and result delivery Model preparation and reuse Package identity, artifact references, named-object retrieval Incremental output and conversation Request state, cancellation, output lifecycle Multi-Provider inference Assignments, dependencies, recovery PreparedModel describes a verified local package. Provider selection and remote preparation follow during invocation. The study will extend focused tests to the selected model, device and recovery configurations.

## Slide 33: DI Collaboration Example: Named Computation Dependencies

Provider A layer group 1 Provider B layer group 2 Provider C output stage activation activation Verify a local model package; choose partitions and Providers after ACK collection closes. Providers authenticate Selection before preparing or reusing assigned resources. Pipeline divides layer groups; tensor partitioning needs supported collective exchanges. ONNX [25] describes the graph; Repo [24] stores artifacts. Neither implements distributed execution by itself. The proposed contribution is checked runtime assignment and data exchange, not model partitioning itself.

## Slide 34: DI State and Planned Validation

Model weights, reusable artifacts, request activations and key/value (K/V) state have different lifetimes. [31] With K/V caching, the runtime reuses prior tensors and supplies next-step inputs; resident weights do not replace this state. Cross-process tests will compare distributed results with the same model's unsplit execution. Begin with one supported model/partition; evaluate additional strategies and LLM streaming as their implementations become ready. Earlier small-model measurements motivate separate preparation, transfer and execution measurements in the new campaigns. [28]

## Slide 35: Cross-Application Study: Where Does the Framework Stop?

Core checks Application responsibility Signer, assigned role and edge UAV: view/time requirements; DI: model/tensor compatibility Complete object; permitted reuse Decide whether the image or retained state remains useful Cancellation/replacement state Stop physical work or drain computation and release resources Test both applications with wrong-role inputs and authentic but application-invalid inputs. Record which layer rejects them; retain valid controls. Expected outcome: shared checks and explicit application validators. Signatures establish provenance, not image truth or numerical correctness.

## Slide 36: Validation Stages: What Each Test Establishes

Stage Evidence sought Contract / local tests Valid transitions and explicit rejection cases MiniNDN [22] Authenticated request chains, caches, loss and outages UAV simulation / bench Operator workflow, telemetry, camera and command handling; PX4 SITL [23] is simulation TigerCluster [28] (preparation) Multi-node GPU inference: correctness, latency, data-transfer overhead and peak GPU memory Supported devices Actual camera/inference outputs, memory and processing costs Use defined outcomes and reproducible configurations. Cluster experiments complement UAV wireless and mobility tests; simulation does not establish real-flight safety.

## Slide 37: RQ3: Define the Baselines Before Comparing

NSC [10]: named service calls; describe the tested executor, timeout, queue and recovery behavior. gRPC [17]: describe resolver, sequential/parallel policy, deadlines, retry and cancellation. Match Provider count, capacity, task, topology/traces, publication times and measurement windows. Report fixed-list and dynamically discovered configurations as different configurations. Specify primary outcomes, completion deadlines and useful-improvement criteria before comparative runs. Comparative conclusions will refer to the tested configurations. Historical unmatched results motivate questions for the matched experiments.

## Slide 38: Preliminary Evidence: Mobility and Completion

Ten seeds (62–71), four Providers/system, 2 m/s; mean seed completion rate. [28] AP radius NDNFSF gRPC NSC 50 m 54.57% 53.80% 55.47% 100 m 97.97% 97.97% 98.33% 150 m 100.00% 100.00% 100.00% Observed completion rates are close across systems. The next slide examines switching latency in a separate holdout campaign (seeds 72–81).

## Slide 39: Preliminary Evidence: Switching Latency

Seeds 72–81; four Providers/system; 100 m, 2 m/s; 1,312 aligned switching requests. [28] Comparison Seed-p95 difference 95% CI NDNFSF minus gRPC-SEQ-4 -597 ms [-992, -200] ms NDNFSF minus NSC-4 -2618 ms [-2916, -2315] ms Completed: NDNFSF 1,311; gRPC 1,297; NSC 1,296 (of 1,312 each). NDNFSF has lower mean seed-p95 latency in this switching workload. Each p95 uses that system’s successful requests; different failure sets limit interpretation.

## Slide 40: Preliminary Evidence: Execution Starts

Six-seed, capacity-matched comparison; 1,800 requests per system. [28] Configuration Completed requests Execution starts NDNFSF 1,798 / 1,800 1,800 gRPC-PAR-4 1,800 / 1,800 7,180 NSC-4 also completed 1,800/1,800, with 1.002 starts/request (rounded). No work-efficiency advantage over NSC is claimed. NDNFSF used fewer execution starts than gRPC-PAR-4 at the reported completion rates. Bytes, CPU time and energy require separate measurement.

## Slide 41: Proposed Evaluation: Tests That Decide the Claims

RQ Controlled test Decision / measurement RQ1 One User, four candidates; identical discovery encryption Reject unauthorized starts; compare grant/withdrawal and invocation costs RQ2 Wrong role/edge within one request; direct/cache delivery; individual-check ablations Reject invalid dependencies before dependent work; valid tasks complete RQ2/3 Cancel/replace one Provider Reject late old work after local update; measure recovery RQ3 Fixed inputs; UAV relay and feasible unsplit DI controls Compare correct outputs, completion, latency, bytes and memory Retain signatures and request IDs in ablations. Define tolerances and useful-improvement criteria before runs.

## Slide 42: Remaining Work and Scope Controls

Minimum case Required evidence UAV Two capture roles and one analyzer; fixed images/algorithm; compare direct exchange with User relay DI Target two-Provider YOLO partition; declare crossing tensors; fix model/cut after compatibility checks Both graphs Valid completion, wrong-input rejection, one cancellation/replacement case, matched costs Extensions More models, LLM streaming, larger deployments and real-flight trials These are planned cases. If either core graph cannot be validated, reconsider its claim and schedule with the committee; optional features cannot replace missing core evidence.

## Slide 43: A Bounded Path to Spring 2027

Internal target Required deliverable September 2026 Agree scope/RQs; freeze one UAV graph and one DI partition; resolve correctness blockers October–November 2026 Complete registered experiments; freeze evidence and write bounded RQ answers December 2026 Complete integrated advisor-review draft January 2027 Committee review and readiness decision; reduce scope or revise timing if core evidence is

missing February 2027 Final dissertation defense, subject to committee approval Committee draft: two weeks before defense. Spring 2027 application/candidacy: February 5; final ProQuest: March 26. Recheck before filing. [29,30]

## **Slide 44: Expected Research Outcomes**

RQ1 answer: tradeoff regions across permission overlap, membership changes and withdrawals, using complete alternatives with equal protection. RQ2 answer: which assignment and dependency rules can be shared across UAV and DI, what they reject, and their recovery limits. RQ3 answer: measured benefits, negative results and deployment limits, including which checks belong in Core or the application. Reproducible configurations and evidence linking each conclusion to its tested scope. The intended outcome is a tested service protocol and an evidence-backed account of when its mechanisms are worth their cost.

## **Slide 45: Discussion: Is This Scope Sufficient and Feasible?**

1. Are the three research questions specific enough to support defensible answers? 2. Are the authorization alternatives complete and fairly configured? 3. Are one UAV graph and one supported DI partition sufficient to test the abstraction? 4. Which optional features should be excluded if they delay the core evidence? Committee feedback sought: research significance, bounded scope, evaluation criteria, and a feasible completion plan.

## **Slide 46: Backup: Aggregate Service Permissions**

Within one Controller domain, combine several service permissions in one entity's DKEY; retain identity-signing credentials. [19,28] DNMP represents command authority through role keys and schema rules. One role can permit several commands. [12] Where equivalent permissions require distinct role credentials, NDNSF avoids maintaining those extra signing credentials. Fewer independently managed credentials does not imply fixed DKEY size or lower total cost. Policy size, key distribution and updates still matter.

## **Slide 47: Backup: When Another Authorization Design May Be Better**

Known Providers and simple rules: signed invocation plus recipient encryption may be sufficient. A small, stable receiver group: ordinary group-key management may be attractive. Rich command-argument restrictions: DNMP-style rules make those constraints explicit. [12] Many changing entitlements: measure both policy/key updates and credential/rule updates before claiming a management advantage. Compare complete combinations of authorization and encryption. ABE-backed authorization is one design choice, not a universally superior solution.

## **Slide 48: Backup: Archived Small-Model Diagnostic**

Original proposal: ten successful runs per layout. [32] Layout Mean latency p95 Shared-backbone distributed layout 259.4 ms 283.9 ms Single Provider 186.8 ms 206.3 ms This motivates measuring preparation and transfer costs. Their causal contribution was not isolated; model/export identity and raw runs require verification. Historical record, not evidence for the newer DI design. Further archived observations and limits are retained in the proposal's Appendix A.

## Slide 49: Backup: Provider-Transition Discovery Configurations

Provider-transition configuration Completed NDNF, no pre-registered Provider addresses 357 / 357 Static-list NSC/gRPC configurations 0 / 357 Pre-registered baseline variants 357 / 357 Completion depends on how candidates are configured in these runs. Resolver or discovery integration can update baseline candidate lists. [27,28]

## Slide 50: Backup: Historical Invocation under Packet Loss

Historical per-link loss NDNF completed requests 1% 599 / 600 3% 598 / 600 10% (best recorded run) 539 / 600 Archived NDNF results from a separate campaign. The 10% value is the best recorded run, not an aggregate estimate; these values are not a matched baseline comparison. [28] Per-link packet loss is not the probability of service failure: retrieval retries and the request deadline also affect completion.

## Slide 51: References: NDN and Authorization

[1] L. Zhang et al., “Named Data Networking,” ACM SIGCOMM CCR, 2014. [2] T. Li et al., “A Brief Introduction to NDN Dataset Synchronization (NDN Sync),” MILCOM, 2018. [3] P. Moll et al., “A Brief Introduction to State Vector Sync,” NDN-0073, rev. 2, 2021. [4] Z. Zhang et al., “NAC: Automating Access Control via Named Data,” MILCOM, 2018. [12] K. Nichols, “Lessons Learned Building a Secure Network Measurement Framework using Basic NDN,” ACM ICN, 2019. [16] Y. Yu et al., “Schematizing Trust in Named Data Networking,” ACM ICN, 2015. [19] S. Dulal et al., “Enhancing NAC-ABE to Support Access Control for mHealth Applications and Beyond,” arXiv:2311.07299, 2023. [26] NDN Project, “Data Packet,” NDN Packet Format Specification. docs.named-data.net/NDN-packet-spec/current/data.html (accessed Sept. 2026).

## Slide 52: References: Invocation and Composition

[6] C. Tschudin and M. Sifalakis, “Named Functions and Cached Computations,” IEEE CCNC, 2014. [7] M. Krol and I. Psaras, “NFaaS: Named Function as a Service,” ACM ICN, 2017. [9] M. Krol et al., “RICE: Remote Method Invocation in ICN,” ACM ICN, 2018. [10] D. Meirovitch and L. Zhang, “NSC—Named Service Calls, or a Remote Procedure Call for NDN,” NDN-0074, rev. 1, 2021. [33] M. Krol, S. Mastorakis, D. Oran and D. Kutscher, “Compute First Networking: Distributed Computing Meets ICN,” ACM ICN, pp. 67–77, 2019. DOI: 10.1145/3357150.3357395. [15] C. Barrios, E. Alker and M. Kumar, “CaSCON: Cache-Assisted Service Composition Orchestration Leveraging NDN,” IEEE SMARTCOMP, pp. 122–129, 2025. DOI: 10.1109/SMARTCOMP65954.2025.00061. [17] gRPC Authors, “Core Concepts, Architecture and Lifecycle,” gRPC documentation. [27] gRPC Authors, “Custom Name Resolution,” grpc.io/docs/guides/custom-name-resolution/ (accessed Sept. 2026).

## Slide 53: References: Tools, Evidence and Degree Planning

[22] Mini-NDN, project documentation: minindn.ndn.today. [23] PX4, “Software-in-the-Loop Simulation,” PX4 User Guide. [24] NDN Project, “Repo Protocol Specification,” repo-ng documentation. [25] ONNX, “Open Neural Network Exchange Intermediate Representation Specification.” [28] NDNF, research-revision-audit.md, companion claim/evidence ledger, September 9, 2026. Contains campaign paths and required validation. [29] University of Memphis CS, “Information for Computer Science Ph.D. Students,” forms\_grad\_phd\_programinfo.pdf (accessed September 9, 2026). [30] University of Memphis Graduate School, “Graduate School Calendar,” Spring 2027. memphis.edu/gradschool/news-events/calendar.php (checked September 14, 2026). [31] ONNX Runtime, “How to configure the past

present share buffer,” GenAI documentation (accessed September 11, 2026). [32] T. Ma, archived dissertation proposal, Section 5.6; Tianxing\_Dissertation\_Proposal\_Origin.pdf, complete 59-page version.